Bab 12

Implementasi Aplikasi

Mengubah rancangan menjadi kode yang berjalan.

12

DI BAB INI

- Implementasi class
- Encapsulation
- Inheritance/composition sesuai rancangan
- Polymorphism bila diperlukan
- Module/package
- File handling
- Integrasi antar-class

Pada Bab 11 kita telah merancang Sistem Peminjaman Perpustakaan: siapa penggunanya, data apa yang diperlukan, dan aturan apa yang harus dijaga. Bab ini mengubah rancangan tersebut menjadi aplikasi Python yang dapat dijalankan melalui terminal. Model dan layanan yang dibuat di sini akan digunakan kembali untuk GUI pada Bab 13.

Kita menggunakan kasus yang sama: satu kode buku mewakili satu eksemplar buku cetak atau satu lisensi e-book. Buku cetak dipinjam selama 14 hari, sedangkan e-book selama 7 hari. Satu buku hanya boleh mempunyai satu peminjaman aktif. Riwayat yang sudah selesai tetap disimpan. Aplikasi digunakan oleh satu petugas pada satu komputer; login, denda, dan akses serentak belum termasuk ruang lingkupnya.

Penerapan class dalam proyek utuh membantu melihat hubungan antara data, aturan, dan pekerjaan pengguna, sebagaimana dibahas dalam proyek terpadu oleh [North, 2025]. Kode dalam bab ini disusun untuk rancangan perpustakaan pada Bab 11, dengan pemisahan komponen agar setiap perilaku dapat diperiksa sebelum dihubungkan ke tampilan [Wilson, 2024].

CAPAIAN PEMBELAJARAN

Setelah mempelajari bab ini, Anda diharapkan mampu:

- menerjemahkan rancangan class dan kontrak method menjadi kode Python;
- menjaga identitas, nilai attribute, dan perubahan status melalui encapsulation;
- menerapkan inheritance dan hubungan antar-object sesuai tanggung jawabnya;
- menggunakan polymorphism untuk variasi perilaku yang memang diperlukan;
- menyusun module dalam package dan menyimpan data beserta relasinya ke JSON;
- mengintegrasikan pendaftaran, peminjaman, pengembalian, serta simpan–muat;
- memeriksa hasil normal dan penolakan transaksi tanpa membuka GUI.

CARA MENGIKUTI KODE DALAM BAB INI

Gunakan Python 3.10 atau yang lebih baru. Seluruh contoh hanya memakai pustaka standar Python. Buat folder proyek baru agar tidak bercampur dengan contoh perpustakaan dari bab sebelumnya. Setiap listing yang berjudul nama file merupakan **isi lengkap file tersebut**; setiap

file hanya perlu dibuat sekali. Listing berjudul `demo_...py` adalah pemeriksaan tambahan, bukan bagian yang digabungkan ke `main.py`.
Lengkapi semua file inti sampai Subbab Integrasi Antar-Class sebelum menjalankan contoh yang mengimpor package `perpustakaan`. Urutan pembahasan mengikuti konsep; beberapa module yang diimpor baru dituliskan pada subbab berikutnya.

12.1 Implementasi Class

Menerjemahkan Tanggung Jawab ke Kode

Class tidak cukup hanya mempunyai attribute yang sama dengan diagram. Method-nya harus memenuhi kontrak: menerima masukan yang sesuai, menjaga aturan, lalu menghasilkan nilai atau exception yang dapat ditangani pemanggil. Pembagian pekerjaan berikut mempertahankan rancangan pada Bab 11.

Tabel 12.1. Tanggung jawab komponen pada implementasi.

Komponen	Tanggung jawab
`Buku`, turunannya	Menyimpan kode dan judul, menyediakan lama pinjam sesuai jenis buku.
`Anggota`	Menyimpan ID anggota dan nama yang valid.
`Peminjaman`	Menghubungkan anggota dengan buku, menyimpan tanggal, dan menyelesaikan satu transaksi.
`Perpustakaan`	Mendaftarkan object, mencari data, memeriksa ketersediaan, dan mengoordinasikan transaksi serta simpan–muat.
`PenyimpananJSON`	Membaca dan menulis dictionary melalui file JSON.
`main.py`	Menerima input terminal dan menampilkan hasil layanan.

Kode buku dan ID anggota merupakan identitas tetap. Judul dan nama boleh diperbarui, tetapi tidak boleh kosong. Object `Peminjaman` menyimpan rujukan ke object anggota dan buku yang sudah terdaftar. Saat data disimpan, rujukan tersebut diubah menjadi ID; saat dimuat, hubungan object dibangun kembali.

Menyiapkan Exception Aplikasi

Buat folder `perpustakaan`, lalu file `kesalahan.py` di dalamnya. Semua exception khusus mewarisi `PerpustakaanError`. Dengan demikian, antarmuka dapat menangkap kesalahan aplikasi melalui satu induk, sedangkan pemeriksaan yang lebih khusus tetap dapat membedakan penyebabnya.

perpustakaan/kesalahan.py

```python
class PerpustakaanError(Exception):
    """Induk kesalahan yang dapat ditampilkan oleh antarmuka."""


class DataTidakValidError(PerpustakaanError):
    pass


class DataTidakDitemukanError(PerpustakaanError):
    pass


class BukuSedangDipinjamError(PerpustakaanError):
    pass


class PeminjamanSelesaiError(PerpustakaanError):
    pass


class DataRusakError(PerpustakaanError):
    pass


class PenyimpananError(PerpustakaanError):
    pass
```

`DataTidakValidError` dipakai untuk masukan seperti nama kosong atau tanggal kembali yang terlalu awal. `DataTidakDitemukanError` berarti ID yang diminta belum terdaftar. `BukuSedangDipinjamError` dan `PeminjamanSelesaiError` menyatakan penolakan transaksi. `DataRusakError` menyatakan isi simpanan tidak dapat dipakai, sedangkan `PenyimpananError` menyatakan kegagalan membuka atau menulis file.

Membuat Model Buku, Anggota, dan Peminjaman

Buat file `model.py` dalam folder yang sama. Listing berikut memuat seluruh class domain, termasuk property dan turunan buku yang akan dibahas lebih rinci pada subbab berikutnya. Model tidak memanggil `input()`, `print()`, atau komponen GUI; hasilnya dapat dipakai oleh antarmuka yang berbeda.

perpustakaan/model.py

```python
from abc import ABC, abstractmethod
from datetime import date, datetime, timedelta

from .kesalahan import DataTidakValidError, PeminjamanSelesaiError
```

```
5
6
7  def teks_wajib(nilai, label):
8      if not isinstance(nilai, str) or not nilai.strip():
9          raise DataTidakValidError(f"{label} harus berupa teks yang terisi.")
10     return nilai.strip()
11
12
13 def tanggal_valid(nilai):
14     if not isinstance(nilai, date) or isinstance(nilai, datetime):
15         raise DataTidakValidError("Tanggal harus berupa object date.")
16     return nilai
17
18
19 class Buku(ABC):
20     JENIS = ""
21
22     def __init__(self, kode, judul):
23         self._kode = teks_wajib(kode, "Kode buku")
24         self.judul = judul
25
26     @property
27     def kode(self):
28         return self._kode
29
30     @property
31     def judul(self):
32         return self._judul
33
34     @judul.setter
35     def judul(self, nilai):
36         self._judul = teks_wajib(nilai, "Judul buku")
37
38     @abstractmethod
39     def lama_pinjam(self):
40         """Mengembalikan lama pinjam dalam hari."""
41
42     def ke_data(self):
43         return {"kode": self.kode, "judul": self.judul, "jenis": self.JENIS}
44
45     def __str__(self):
46         return f"[{self.kode}] {self.judul}"
47
48
49 class BukuCetak(Buku):
50     JENIS = "cetak"
```

```
51
52      def lama_pinjam(self):
53          return 14
54
55
56  class EBook(Buku):
57      JENIS = "ebook"
58
59      def lama_pinjam(self):
60          return 7
61
62
63  class Anggota:
64      def __init__(self, id_anggota, nama):
65          self._id_anggota = teks_wajib(id_anggota, "ID anggota")
66          self.nama = nama
67
68      @property
69      def id_anggota(self):
70          return self._id_anggota
71
72      @property
73      def nama(self):
74          return self._nama
75
76      @nama.setter
77      def nama(self, nilai):
78          self._nama = teks_wajib(nilai, "Nama anggota")
79
80      def info(self):
81          return f"{self.id_anggota} - {self.nama}"
82
83      def ke_data(self):
84          return {"id_anggota": self.id_anggota, "nama": self.nama}
85
86
87  class Peminjaman:
88      def __init__(self, id_pinjam, anggota, buku, tanggal, jatuh_tempo=None):
89          if not isinstance(anggota, Anggota) or not isinstance(buku, Buku):
90              raise DataTidakValidError("Peminjaman memerlukan anggota dan buku.")
91          self._id_pinjam = teks_wajib(id_pinjam, "ID peminjaman")
92          self._anggota = anggota
93          self._buku = buku
94          self._tanggal_pinjam = tanggal_valid(tanggal)
95          if jatuh_tempo is None:
```

```
            try:
                jatuh_tempo = tanggal + timedelta(days=buku.lama_pinjam())
            except OverflowError as exc:
                raise DataTidakValidError("Tanggal di luar jangkauan.") from
    exc
        self._tanggal_jatuh_tempo = tanggal_valid(jatuh_tempo)
        if jatuh_tempo < tanggal:
            raise DataTidakValidError("Jatuh tempo mendahului tanggal pinjam.")
        self._tanggal_kembali = None

    @property
    def id_pinjam(self):
        return self._id_pinjam

    @property
    def anggota(self):
        return self._anggota

    @property
    def buku(self):
        return self._buku

    @property
    def tanggal_pinjam(self):
        return self._tanggal_pinjam

    @property
    def tanggal_jatuh_tempo(self):
        return self._tanggal_jatuh_tempo

    @property
    def tanggal_kembali(self):
        return self._tanggal_kembali

    @property
    def aktif(self):
        return self.tanggal_kembali is None

    def kembalikan(self, tanggal):
        if not self.aktif:
            raise PeminjamanSelesaiError("Peminjaman sudah selesai.")
        tanggal = tanggal_valid(tanggal)
        if tanggal < self.tanggal_pinjam:
            raise DataTidakValidError("Pengembalian mendahului tanggal pinjam.")

        self._tanggal_kembali = tanggal
```

```
140
141     def ke_data(self):
142         kembali = self.tanggal_kembali
143         return {
144             "id_pinjam": self.id_pinjam,
145             "id_anggota": self.anggota.id_anggota,
146             "kode_buku": self.buku.kode,
147             "tanggal_pinjam": self.tanggal_pinjam.isoformat(),
148             "tanggal_jatuh_tempo": self.tanggal_jatuh_tempo.isoformat(),
149             "tanggal_kembali": kembali.isoformat() if kembali else None,
150         }
```

Fungsi `teks_wajib()` memeriksa tipe dan isi teks, kemudian membuang spasi di awal serta akhir. Fungsi `tanggal_valid()` memastikan nilai berupa `date`, bukan teks atau object `datetime`. Pemeriksaan ini menjaga agar operasi tanggal di model mempunyai arti yang sama. Input dari terminal akan dikonversi dengan `date.fromisoformat()` sebelum diteruskan ke layanan.

Parameter `jatuh_tempo=None` pada constructor `Peminjaman` mempunyai dua kegunaan. Untuk transaksi baru, nilainya dihitung dari tanggal pinjam ditambah hasil `buku.lama_pinjam()`. Saat memuat riwayat dari JSON, pemanggil memberikan tanggal jatuh tempo yang sudah tersimpan. Karena itu, perubahan kebijakan lama pinjam kelak tidak mengubah kesepakatan transaksi lama.

Property `aktif` dihitung dari `tanggal_kembali is None`. Tidak ada attribute boolean kedua yang harus diperbarui bersamaan. Method `kembalikan()` memeriksa keadaan dan tanggal terlebih dahulu; tanggal kembali baru diisi jika seluruh syarat terpenuhi. Method `ke_data()` menyiapkan nilai yang bisa ditulis sebagai JSON, tanpa melakukan pekerjaan file di dalam model.

SATU ATURAN, SATU TEMPAT PEMERIKSAAN

Model memeriksa kebenaran satu object. Layanan memeriksa aturan yang melibatkan koleksi object. Misalnya, `Peminjaman` memeriksa tanggal kembali, tetapi hanya `Perpustakaan` yang dapat memeriksa apakah buku sudah mempunyai peminjaman aktif lain.

12.2 Encapsulation

Membatasi Perubahan melalui Antarmuka Object

Encapsulation pada proyek ini menjaga agar perubahan data mengikuti aturan yang sudah ditetapkan. Property `kode` dan `id_anggota` hanya mempunyai getter sehingga tidak dapat diberi nilai melalui antarmuka publik. Ini diperlukan karena kode dan ID dipakai sebagai kunci dictionary serta penghubung riwayat. Jika identitas berubah sepihak, kunci koleksi dan data object dapat tidak sesuai.

Property `judul` dan `nama` mempunyai setter karena keduanya boleh dikoreksi. Setter memanggil validasi sebelum mengisi attribute internal; jika nilai baru ditolak, nilai lama tetap tersimpan. Tanggal dan rujukan pada peminjaman hanya dibaca melalui property. Penyelesaian transaksi dilakukan dengan method `kembalikan()`, bukan dengan mengisi tanggal kembali secara langsung.

Setelah seluruh file inti tersedia, buat file `demo_encapsulation.py` di folder utama proyek. Isi dengan contoh berikut, lalu jalankan melalui terminal.

Perintah menjalankan demo

```
python demo_encapsulation.py
```

demo_encapsulation.py

```
from datetime import date

from perpustakaan import Anggota, BukuCetak, Peminjaman
from perpustakaan.kesalahan import DataTidakValidError, PeminjamanSelesaiError


buku = BukuCetak("B001", "Dasar OOP")
buku.judul = "OOP dengan Python"
print(buku)
try:
    buku.judul = "   "
except DataTidakValidError as exc:
    print("Ditolak:", exc)
print("Judul tetap:", buku.judul)
try:
    buku.kode = "B999"
except AttributeError:
    print("Kode hanya dapat dibaca.")

ani = Anggota("M001", "Ani")
pinjam = Peminjaman("P001", ani, buku, date(2026, 10, 27))
print("Aktif:", pinjam.aktif)
pinjam.kembalikan(date(2026, 11, 2))
print("Aktif setelah kembali:", pinjam.aktif)
try:
    pinjam.kembalikan(date(2026, 11, 3))
except PeminjamanSelesaiError as exc:
    print("Ditolak:", exc)
```

Keluaran

```
[B001] OOP dengan Python
Ditolak: Judul buku harus berupa teks yang terisi.
Judul tetap: OOP dengan Python
Kode hanya dapat dibaca.
Aktif: True
Aktif setelah kembali: False
Ditolak: Peminjaman sudah selesai.
```

Judul berhasil berubah karena nilai barunya valid. Percobaan mengisi judul dengan spasi ditolak sebelum `_judul` diubah. Pemberian nilai ke `buku.kode` memicu `AttributeError` karena property tersebut tidak mempunyai setter. Pemanggilan pengembalian kedua ditolak karena transaksi sudah selesai.

Menjaga Koleksi dan Memahami Batasnya

Layanan menyimpan buku, anggota, dan peminjaman dalam dictionary internal. Method daftar mengembalikan list baru. Menghapus isi list hasil tersebut tidak menghapus catatan di layanan. Namun, object di dalam list masih merupakan object yang sama, sehingga perubahan melalui setter yang sah tetap terlihat pada layanan.

Salinan koleksi tersebut adalah *shallow copy*, bukan salinan mendalam seluruh object. Awalan satu garis bawah juga merupakan konvensi untuk pemakaian internal, bukan mekanisme keamanan. Pemanggil yang sengaja menulis ke `_judul` atau koleksi internal dapat melewati aturan. Karena itu, kode antarmuka pada proyek ini selalu memakai property dan method publik yang disediakan.

PEMBUATAN OBJECT DAN TRANSAKSI APLIKASI

Pembuatan `Peminjaman` secara langsung pada contoh konsep hanya memeriksa aturan satu catatan. Dalam aplikasi, lakukan peminjaman melalui `perpus.pinjamkan(...)` agar keberadaan anggota, keberadaan buku, dan larangan peminjaman ganda ikut diperiksa. Demikian juga, lakukan pengembalian melalui layanan agar pemanggil bekerja dengan catatan yang terdaftar.

12.3 Inheritance/Composition sesuai Rancangan

Inheritance untuk Jenis Buku

`BukuCetak` dan `EBook` merupakan jenis `Buku`. Keduanya memakai validasi kode, setter judul, method `ke_data()`, dan representasi teks yang diwarisi dari induknya. Perbedaannya berada pada nilai `JENIS` dan implementasi `lama_pinjam()`.

Class `Buku` menggunakan `ABC` dan `@abstractmethod`. Kita tidak membuat `Buku(...)` secara langsung karena lama pinjamnya belum ditentukan. Class turunan yang konkret wajib menyediakan implementasi method abstrak tersebut. Pemilihan inheritance didasarkan pada kesamaan kontrak dan hubungan jenis, sebagaimana pembahasan pe-

makaian class oleh [Lutz, 2025].

Association untuk Rujukan, Composition untuk Catatan Layanan

`Peminjaman` bukan turunan `Anggota` ataupun `Buku`. Ia menyimpan rujukan ke keduanya melalui attribute internal. Hubungan ini adalah association: satu peminjaman menunjuk satu peminjam dan satu buku, sedangkan object anggota serta buku dapat dipakai oleh banyak catatan sepanjang riwayatnya.

Simpan contoh berikut sebagai `demo_relasi.py`. Operator `is` memeriksa apakah dua rujukan menunjuk object yang sama, bukan sekadar memiliki isi yang sama.

demo_relasi.py

```
from datetime import date

from perpustakaan import Anggota, Buku, BukuCetak, Peminjaman


ani = Anggota("M001", "Ani")
buku = BukuCetak("B001", "Dasar OOP")
pinjam = Peminjaman("P001", ani, buku, date(2026, 10, 27))

print("BukuCetak adalah Buku:", isinstance(buku, Buku))
print("Anggota yang sama:", pinjam.anggota is ani)
print("Buku yang sama:", pinjam.buku is buku)
ani.nama = "Ani Lestari"
print("Peminjam:", pinjam.anggota.nama)
```

Keluaran

```
BukuCetak adalah Buku: True
Anggota yang sama: True
Buku yang sama: True
Peminjam: Ani Lestari
```

Nama baru terlihat melalui `pinjam.anggota.nama` karena catatan menyimpan rujukan ke object `ani` yang sama. Nama anggota tidak digandakan di dalam peminjaman. Konsekuensinya, riwayat menampilkan nama anggota terbaru; menyimpan salinan nama pada waktu transaksi merupakan kebutuhan berbeda yang belum diterapkan pada proyek ini.

Dalam rancangan Bab 11, layanan `Perpustakaan` memiliki catatan peminjamannya sebagai composition. Implementasinya dilakukan dengan membuat `Peminjaman` di dalam `pinjamkan()` dan menyimpannya dalam dictionary layanan. Buku dan anggota didaftarkan sebagai object yang sudah dibuat, sesuai hubungan aggregation pada diagram. Composition di sini menyatakan kepemilikan dalam model aplikasi; menutup program tidak berarti menghapus riwayat yang sudah disimpan ke file.

Object penyimpanan diberikan ke constructor layanan. Class `Perpustakaan`

memakainya tanpa mewarisi class file JSON. Pemilihan hubungan ini mengikuti tanggung jawab object: layanan mengelola transaksi, sedangkan object penyimpanan mengurus file.

12.4 Polymorphism bila Diperlukan

Polymorphism diperlukan ketika pemanggil menjalankan operasi yang sama tetapi perilakunya dapat berbeda menurut jenis object. Pada aplikasi ini, perbedaannya nyata: buku cetak mempunyai masa pinjam 14 hari dan e-book 7 hari. Constructor `Peminjaman` cukup memanggil `buku.lama_pinjam()`, tanpa memilih rumus berdasarkan jenis buku.

Simpan dan jalankan `demo_polymorphism.py` berikut. Kedua object melewati baris pembuatan peminjaman yang sama, tetapi menghasilkan tanggal jatuh tempo berbeda.

demo_polymorphism.py

```
from datetime import date

from perpustakaan import Anggota, BukuCetak, EBook, Peminjaman


ani = Anggota("M001", "Ani")
koleksi = [BukuCetak("B001", "Dasar OOP"), EBook("E001", "Dasar Python")]
for nomor, buku in enumerate(koleksi, start=1):
    pinjam = Peminjaman(f"P{nomor:03d}", ani, buku, date(2026, 10, 27))
    print(buku.kode, buku.lama_pinjam(), "hari, jatuh tempo",
          pinjam.tanggal_jatuh_tempo)
```

Keluaran

```
B001 14 hari, jatuh tempo 2026-11-10
E001 7 hari, jatuh tempo 2026-11-03
```

Jika kelak ditambahkan jenis buku lain dengan masa pinjam berbeda, turunan baru menyediakan `lama_pinjam()`. Rumus di constructor peminjaman tetap dapat digunakan. Namun, integrasi jenis baru juga memerlukan pembaruan pilihan input dan pemetaan jenis pada pemuatan JSON. Polymorphism mengurangi percabangan pada perhitungan transaksi, bukan menghilangkan seluruh pekerjaan integrasi.

Kontrak `Penyimpanan` juga memberi peluang polymorphism: layanan memanggil `muat()` dan `simpan(data)` tanpa mengetahui cara penulisan file. Bab ini menyediakan satu implementasi, yaitu JSON. Implementasi lain dapat ditambahkan jika kebutuhan memang berubah; tidak perlu membuat banyak class penyimpanan hanya untuk memperbanyak turunan.

12.5 Module/Package

Menyusun Folder dan Arah Import

Module adalah satu file Python, sedangkan package mengelompokkan module yang berkaitan. Pada proyek ini kita memakai package biasa dengan file `__init__.py`. Struktur akhirnya sebagai berikut. Folder `data` dan file JSON dibuat ketika penyimpanan pertama berhasil.

Struktur folder proyek

```
proyek_perpustakaan/
|-- main.py
|-- demo_integrasi.py
|-- demo_encapsulation.py
|-- demo_relasi.py
|-- demo_polymorphism.py
|-- cek_kontrak.py
|-- perpustakaan/
|   |-- __init__.py
|   |-- kesalahan.py
|   |-- model.py
|   |-- penyimpanan.py
|   `-- layanan.py
`-- data/
    `-- perpustakaan.json
```

Tabel 12.2. Isi file dan ketergantungannya.

File	Isi	Memakai
`kesalahan.py`	Exception khusus	Exception bawaan Python
`model.py`	Class domain dan validasinya	Exception khusus, tanggal, ABC
`penyimpanan.py`	Kontrak dan adapter JSON	Exception khusus, JSON, path, file sementara
`layanan.py`	Koordinasi aplikasi	Model, kontrak penyimpanan, exception
`__init__.py`	Nama publik package	Model, layanan, penyimpanan, exception
`main.py`	Menu terminal	Nama publik package dan pustaka standar

Model tidak mengimpor layanan; penyimpanan tidak mengimpor menu; layanan tidak mengimpor `main.py`. Arah import tersebut mencegah ketergantungan berputar

dan memungkinkan model digunakan kembali ketika menu terminal diganti dengan GUI. Pemisahan komponen berdasarkan pekerjaan yang jelas juga memudahkan pemeriksaan perilaku setiap komponen [Wilson, 2024].

Menetapkan Antarmuka Package

Buat `perpustakaan/__init__.py` dengan isi berikut. File ini menyediakan nama yang paling sering dipakai oleh antarmuka. Exception khusus yang lebih rinci tetap dapat diimpor dari module `perpustakaan.kesalahan`.

perpustakaan/__init__.py

```
from .model import Anggota, Buku, BukuCetak, EBook, Peminjaman
from .layanan import Perpustakaan
from .penyimpanan import PenyimpananJSON
from .kesalahan import PerpustakaanError
```

Tanda titik pada `from .model import ...` berarti import relatif dari package yang sama. Sebaliknya, `main.py` memakai `from perpustakaan import ...` karena berada di luar package. Jalankan aplikasi dari folder `proyek_perpustakaan`, bukan dengan menjalankan `layanan.py` secara langsung.

Perintah di terminal

```
python main.py
python demo_integrasi.py
```

Jika sistem menyediakan perintah `python3` atau `py` untuk interpreter yang digunakan, sesuaikan nama perintahnya. Tidak ada perintah `pip install` yang diperlukan untuk proyek ini. Import package hanya menyiapkan class; program menu dijalankan oleh blok `if __name__ == "__main__"` dalam `main.py`.

12.6 File Handling

Mengubah Object Menjadi Data JSON

JSON tidak menyimpan object Python beserta method dan rujukannya secara langsung. Karena itu, setiap model menyediakan `ke_data()` yang menghasilkan dictionary berisi teks, angka, atau `None`. Tanggal diubah menjadi teks ISO seperti `2026-10-27`. Nilai `None` menjadi `null` di JSON.

Satu data aplikasi memuat `versi`, nomor peminjaman berikutnya, daftar buku, daftar anggota, dan daftar peminjaman. Penanda versi membuat format yang belum didukung dapat ditolak secara jelas. Nomor berikutnya disimpan agar ID transaksi tetap berlanjut setelah aplikasi ditutup.

Contoh satu catatan peminjaman dalam JSON

```
{
  "id_pinjam": "P001",
  "id_anggota": "M001",
  "kode_buku": "B001",
  "tanggal_pinjam": "2026-10-27",
  "tanggal_jatuh_tempo": "2026-11-10",
  "tanggal_kembali": null
}
```

Catatan tersebut menyimpan identitas anggota dan buku, bukan salinan object atau hasil `id()`. Ketika dimuat, buku dan anggota harus dibuat terlebih dahulu. Layanan kemudian mencari object yang sesuai dengan ID pada catatan. Dengan urutan itu, peminjaman kembali terhubung ke object yang terdaftar, bukan ke salinan terpisah.

Membuat Adapter Penyimpanan JSON

Buat `perpustakaan/penyimpanan.py` berikut. `Protocol` menyatakan bentuk operasi yang diperlukan layanan. Anotasi tersebut membantu pembacaan kode dan pemeriksaan tipe; Python tidak otomatis memeriksa kepatuhan object pada saat runtime hanya karena parameter diberi anotasi.

perpustakaan/penyimpanan.py

```python
import json
import os
import tempfile
from pathlib import Path
from typing import Protocol

from .kesalahan import DataRusakError, PenyimpananError


class Penyimpanan(Protocol):
    def muat(self) -> dict: ...
    def simpan(self, data: dict) -> None: ...


class PenyimpananJSON:
    def __init__(self, path):
        self.path = Path(path)

    def muat(self):
        try:
            with self.path.open(encoding="utf-8") as sumber:
                data = json.load(sumber)
        except FileNotFoundError:
```

```
            raise  # layanan menentukan perilaku saat belum ada file
        except (json.JSONDecodeError, UnicodeError) as exc:
            raise DataRusakError("File JSON tidak dapat dibaca.") from exc
        except OSError as exc:
            raise PenyimpananError("Gagal membuka file data.") from exc
        if not isinstance(data, dict):
            raise DataRusakError("Isi utama JSON harus berupa object.")
        return data

    def simpan(self, data):
        sementara = None
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            with tempfile.NamedTemporaryFile(
                mode="w", encoding="utf-8", dir=self.path.parent,
                prefix=".perpustakaan-", suffix=".tmp", delete=False,
            ) as tujuan:
                sementara = Path(tujuan.name)
                json.dump(data, tujuan, ensure_ascii=False, indent=2)
            os.replace(sementara, self.path)
        except (OSError, TypeError, ValueError) as exc:
            raise PenyimpananError("Gagal menyimpan data.") from exc
        finally:
            if sementara is not None:
                try:
                    sementara.unlink(missing_ok=True)
                except OSError:
                    pass  # pembersihan tidak menutupi kegagalan utama
```

Method `muat()` bertugas membaca JSON dan memastikan nilai utamanya berupa dictionary. Pemeriksaan makna data, seperti ID anggota yang tidak ditemukan atau dua transaksi aktif untuk satu buku, menjadi tugas layanan. File yang belum ada dibiarkan menghasilkan `FileNotFoundError`; layanan akan menganggapnya sebagai pemakaian pertama. File yang ada tetapi rusak menghasilkan exception yang berbeda.

Saat menyimpan, program menulis file sementara pada folder yang sama, menutupnya, lalu memanggil `os.replace()` untuk mengganti file tujuan. Cara ini mengurangi risiko file lama terpotong ketika proses penulisan belum selesai. Pada Windows, file sementara perlu ditutup sebelum diganti. Bagian `finally` membersihkan file sementara yang tersisa tanpa menutupi penyebab kegagalan utama.

BEDAKAN FILE BELUM ADA DAN FILE RUSAK

File yang belum ada adalah keadaan wajar pada pemakaian pertama. File rusak harus dilaporkan agar pengguna dapat memeriksa atau memulihkan datanya. Aplikasi terminal pada bab ini berhenti jika pemuatan gagal; aplikasi tidak melanjutkan dengan data kosong lalu menimpa

simpanan lama.
Penggantian melalui file sementara bukan sistem pencadangan, bukan pengaturan akses serentak, dan tidak menjamin pemulihan dari setiap gangguan listrik atau kerusakan disk. Proyek ini tetap mengikuti batas satu petugas dan satu proses aplikasi.

Memuat Data tanpa Mengubah Keadaan Sebagian

Layanan tidak langsung mengosongkan koleksi ketika mulai memuat file. Ia membuat keadaan sementara, memeriksa seluruh buku, anggota, dan transaksi, lalu mengganti keadaan lama hanya setelah semua data sah. Jika satu rujukan tidak ditemukan atau satu tanggal salah, proses dibatalkan dan keadaan layanan yang lama tetap ada.

Tanggal jatuh tempo dibaca dari file dan diberikan ke constructor peminjaman. Tanggal kembali yang bukan `null` diterapkan melalui `kembalikan()` agar pemeriksaan tanggal dan status tetap digunakan. Setelah itu, layanan memeriksa ID peminjaman, larangan dua catatan aktif pada satu buku, serta urutan nomor berikutnya. Implementasi lengkap proses ini berada dalam `layanan.py` pada subbab berikut.

12.7 Integrasi Antar-Class

Menghubungkan Aturan melalui Layanan

Layanan menjadi pintu masuk operasi aplikasi. Menu tidak mencari sendiri transaksi aktif lalu mengubah attribute peminjaman. Menu memanggil `pinjamkan()` atau `kembalikan()` dan menangani hasilnya. Dengan begitu, GUI pada Bab 13 dapat menggunakan aturan yang sama.

Buat `perpustakaan/layanan.py` dengan isi lengkap berikut. Koleksi buku, anggota, dan peminjaman memakai dictionary dengan identitas sebagai kunci. List untuk tampilan dibuat melalui method daftar. JSON tetap memakai daftar record karena format penyimpanan tidak harus meniru bentuk koleksi internal secara persis.

perpustakaan/layanan.py

```python
from datetime import date

from .kesalahan import (
    PerpustakaanError, DataTidakValidError, DataTidakDitemukanError,
    BukuSedangDipinjamError, DataRusakError,
)
from .model import Anggota, Buku, BukuCetak, EBook, Peminjaman, teks_wajib
from .penyimpanan import Penyimpanan


class Perpustakaan:
    def __init__(self, penyimpanan: Penyimpanan):
```

```
        self._penyimpanan = penyimpanan
        self._buku = {}
        self._anggota = {}
        self._peminjaman = {}
        self._nomor_pinjam_berikutnya = 1

    def tambah_buku(self, buku):
        if not isinstance(buku, Buku):
            raise DataTidakValidError("Data harus berupa object Buku.")
        if buku.kode in self._buku:
            raise DataTidakValidError("Kode buku sudah terdaftar.")
        self._buku[buku.kode] = buku

    def tambah_anggota(self, anggota):
        if not isinstance(anggota, Anggota):
            raise DataTidakValidError("Data harus berupa object Anggota.")
        if anggota.id_anggota in self._anggota:
            raise DataTidakValidError("ID anggota sudah terdaftar.")
        self._anggota[anggota.id_anggota] = anggota

    def _cari(self, koleksi, identitas, label):
        identitas = teks_wajib(identitas, label)
        if identitas not in koleksi:
            raise DataTidakDitemukanError(f"{label} tidak ditemukan.")
        return koleksi[identitas]

    def tersedia(self, kode_buku):
        buku = self._cari(self._buku, kode_buku, "Buku")
        return not any(
            p.aktif and p.buku.kode == buku.kode
            for p in self._peminjaman.values()
        )

    def pinjamkan(self, id_anggota, kode_buku, tanggal):
        anggota = self._cari(self._anggota, id_anggota, "Anggota")
        buku = self._cari(self._buku, kode_buku, "Buku")
        if not self.tersedia(buku.kode):
            raise BukuSedangDipinjamError("Buku masih dipinjam.")
        identitas = f"P{self._nomor_pinjam_berikutnya:03d}"
        pinjam = Peminjaman(identitas, anggota, buku, tanggal)
        self._peminjaman[identitas] = pinjam
        self._nomor_pinjam_berikutnya += 1
        return pinjam

    def kembalikan(self, id_pinjam, tanggal):
        pinjam = self._cari(self._peminjaman, id_pinjam, "Peminjaman")
```

```
        pinjam.kembalikan(tanggal)

    def daftar_buku(self):
        return list(self._buku.values())

    def daftar_anggota(self):
        return list(self._anggota.values())

    def daftar_peminjaman(self, aktif_saja=False):
        return [p for p in self._peminjaman.values()
                if not aktif_saja or p.aktif]

    def simpan(self):
        data = {
            "versi": 1,
            "nomor_pinjam_berikutnya": self._nomor_pinjam_berikutnya,
            "buku": [b.ke_data() for b in self._buku.values()],
            "anggota": [a.ke_data() for a in self._anggota.values()],
            "peminjaman": [p.ke_data() for p in self._peminjaman.values()],
        }
        self._penyimpanan.simpan(data)

    def muat(self):
        try:
            data = self._penyimpanan.muat()
        except FileNotFoundError:
            return  # pertama kali dipakai: belum ada data di disk

        # Susun keadaan sementara; keadaan lama tetap utuh jika data salah.
        try:
            wajib = {"versi", "nomor_pinjam_berikutnya", "buku",
                     "anggota", "peminjaman"}
            if not isinstance(data, dict) or set(data) != wajib:
                raise DataTidakValidError("Field data tidak lengkap.")
            if type(data["versi"]) is not int or data["versi"] != 1:
                raise DataTidakValidError("Versi data tidak didukung.")
            for nama in ("buku", "anggota", "peminjaman"):
                if not isinstance(data[nama], list):
                    raise DataTidakValidError("Koleksi harus berupa list.")
            baru = Perpustakaan(self._penyimpanan)
            jenis_buku = {"cetak": BukuCetak, "ebook": EBook}
            for d in data["buku"]:
                kelas = jenis_buku[d["jenis"]]
                baru.tambah_buku(kelas(d["kode"], d["judul"]))
            for d in data["anggota"]:
                baru.tambah_anggota(Anggota(d["id_anggota"], d["nama"]))
```

```
105             nomor_terbesar = 0
106             for d in data["peminjaman"]:
107                 p = Peminjaman(
108                     d["id_pinjam"], baru._anggota[d["id_anggota"]],
109                     baru._buku[d["kode_buku"]],
110                     date.fromisoformat(d["tanggal_pinjam"]),
111                     date.fromisoformat(d["tanggal_jatuh_tempo"]),
112                 )
113                 if d["tanggal_kembali"] is not None:
114                     p.kembalikan(date.fromisoformat(d["tanggal_kembali"]))
115                 identitas = p.id_pinjam
116                 if (not identitas.startswith("P")
117                         or not identitas[1:].isdigit()
118                         or identitas in baru._peminjaman):
119                     raise DataTidakValidError("ID peminjaman tidak sah.")
120                 if p.aktif and not baru.tersedia(p.buku.kode):
121                     raise DataTidakValidError("Buku dipinjam dua kali.")
122                 baru._peminjaman[identitas] = p
123                 nomor_terbesar = max(nomor_terbesar, int(identitas[1:]))
124             nomor = data["nomor_pinjam_berikutnya"]
125             if type(nomor) is not int or nomor <= nomor_terbesar:
126                 raise DataTidakValidError("Penomoran peminjaman tidak sah.")
127             baru._nomor_pinjam_berikutnya = nomor
128         except (KeyError, TypeError, ValueError, PerpustakaanError) as exc:
129             raise DataRusakError("Struktur data perpustakaan tidak sah.") from
    exc
130
131         self._buku = baru._buku
132         self._anggota = baru._anggota
133         self._peminjaman = baru._peminjaman
134         self._nomor_pinjam_berikutnya = baru._nomor_pinjam_berikutnya
```

Method `pinjamkan()` mencari anggota dan buku yang terdaftar, memeriksa ketersediaan, lalu membentuk catatan. Penambahan ke dictionary dan kenaikan nomor dilakukan setelah constructor berhasil. Karena itu, permintaan yang ditolak tidak menambah riwayat dan tidak menghabiskan nomor transaksi.

Method `tersedia()` menghitung status dari peminjaman aktif. Setelah tanggal kembali terisi, catatan tetap berada di dictionary sebagai riwayat, tetapi tidak lagi menghalangi peminjaman berikutnya. Kita tidak perlu menyelaraskan flag `dipinjam` pada buku dengan flag `aktif` pada transaksi.

Pada `muat()`, pemetaan `jenis_buku` mengubah penanda `"cetak"` atau `"ebook"` menjadi class pembentuk object. Percabangan saat membaca format file berbeda tujuannya dari perhitungan lama pinjam. Setelah object terbentuk, transaksi kembali memakai polymorphism melalui `lama_pinjam()`.

Membuat Menu Terminal

Buat `main.py` di folder utama proyek. Path data dibangun dari lokasi file program, sehingga tidak bergantung pada folder kerja terminal. Menu menerima input, mengubah teks tanggal menjadi `date`, dan meneruskan pekerjaan ke layanan.

main.py

```python
from datetime import date
from pathlib import Path

from perpustakaan import (
    Anggota, BukuCetak, EBook, Perpustakaan,
    PenyimpananJSON, PerpustakaanError,
)


def main():
    path = Path(__file__).resolve().parent / "data" / "perpustakaan.json"
    perpus = Perpustakaan(PenyimpananJSON(path))
    try:
        perpus.muat()
    except PerpustakaanError as exc:
        print("Data tidak dimuat:", exc)
        return 1  # tidak melanjutkan dengan data kosong lalu menimpa file

    while True:
        print("\n1 Buku | 2 Tambah buku | 3 Tambah anggota")
        print("4 Pinjam | 5 Kembali | 6 Riwayat | 0 Simpan dan keluar")
        try:
            pilihan = input("Pilihan: ").strip()
            if pilihan == "1":
                for buku in perpus.daftar_buku():
                    status = "tersedia" if perpus.tersedia(buku.kode) else "
    dipinjam"
                    print(buku, "-", status)
            elif pilihan == "2":
                kode = input("Kode buku: ")
                judul = input("Judul: ")
                jenis = input("Jenis (cetak/ebook): ").strip().lower()
                kelas = {"cetak": BukuCetak, "ebook": EBook}.get(jenis)
                if kelas is None:
                    raise ValueError("Jenis harus cetak atau ebook.")
                perpus.tambah_buku(kelas(kode, judul))
                print("Buku terdaftar.")
            elif pilihan == "3":
                identitas = input("ID anggota: ")
                nama = input("Nama: ")
                perpus.tambah_anggota(Anggota(identitas, nama))
```

```
                print("Anggota terdaftar.")
            elif pilihan == "4":
                anggota = input("ID anggota: ")
                buku = input("Kode buku: ")
                tanggal = date.fromisoformat(input("Tanggal YYYY-MM-DD: ").
    strip())
                pinjam = perpus.pinjamkan(anggota, buku, tanggal)
                print("Berhasil:", pinjam.id_pinjam,
                      "jatuh tempo", pinjam.tanggal_jatuh_tempo)
            elif pilihan == "5":
                identitas = input("ID peminjaman: ")
                tanggal = date.fromisoformat(input("Tanggal YYYY-MM-DD: ").
    strip())
                perpus.kembalikan(identitas, tanggal)
                print("Pengembalian tercatat.")
            elif pilihan == "6":
                for p in perpus.daftar_peminjaman():
                    status = "aktif" if p.aktif else "selesai"
                    print(p.id_pinjam, p.anggota.nama, p.buku.kode, status)
            elif pilihan == "0":
                perpus.simpan()
                print("Data tersimpan.")
                return 0
            else:
                print("Pilihan tidak dikenal.")
        except (PerpustakaanError, ValueError) as exc:
            print("Ditolak:", exc)
        except (EOFError, KeyboardInterrupt):
            print("\nInput dihentikan; perubahan sesi ini belum disimpan.")
            return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Jalankan `python main.py`, lalu daftarkan B001 sebagai buku cetak berjudul Dasar OOP dan M001 sebagai anggota bernama Ani. Gunakan menu Pinjam dengan tanggal `2026-10-27`. Contoh berikut merupakan cuplikan interaksi; tampilan menu yang berulang tidak dituliskan kembali.

Cuplikan interaksi terminal

```
Pilihan: 4
ID anggota: M001
Kode buku: B001
Tanggal YYYY-MM-DD: 2026-10-27
Berhasil: P001 jatuh tempo 2026-11-10
```

```
Pilihan: 1
[B001] Dasar OOP – dipinjam

Pilihan: 0
Data tersimpan.
```

Perubahan selama menu berjalan berada di memori. Pilihan `0` menyimpan data dan baru keluar jika penyimpanan berhasil. Jika penyimpanan ditolak, program menampilkan pesan dan kembali ke menu sehingga pengguna dapat mencoba lagi. Penghentian dengan Ctrl+C atau berakhirnya input tidak menyimpan perubahan sesi secara otomatis. Buka kembali program untuk memeriksa bahwa data yang berhasil disimpan tetap terbaca.

Memeriksa Alur Terpadu secara Berulang

Menu berguna untuk mencoba aplikasi sebagai pengguna, tetapi pengulangan input manual mudah keliru. Buat `demo_integrasi.py` berikut untuk memeriksa satu alur tetap: daftar buku dan anggota, pinjam, tolak peminjaman kedua, kembalikan, pinjam e-book, simpan, lalu muat ulang.

Contoh memakai `TemporaryDirectory` agar setiap eksekusi dimulai dari keadaan baru. File demo berada di folder sementara dan dibersihkan saat blok selesai; file data milik `main.py` tidak digunakan.

demo_integrasi.py

```python
from datetime import date
from pathlib import Path
from tempfile import TemporaryDirectory

from perpustakaan import Anggota, BukuCetak, EBook, Perpustakaan,
    PenyimpananJSON
from perpustakaan.kesalahan import BukuSedangDipinjamError


with TemporaryDirectory() as folder:
    simpanan = PenyimpananJSON(Path(folder) / "perpustakaan.json")
    perpus = Perpustakaan(simpanan)
    perpus.muat()
    perpus.tambah_buku(BukuCetak("B001", "Dasar OOP"))
    perpus.tambah_buku(EBook("E001", "Dasar Python"))
    perpus.tambah_anggota(Anggota("M001", "Ani"))

    pinjam = perpus.pinjamkan("M001", "B001", date(2026, 10, 27))
    print("Pinjam:", pinjam.id_pinjam, "jatuh tempo", pinjam.
    tanggal_jatuh_tempo)
```

```
    print("B001 tersedia:", perpus.tersedia("B001"))
    try:
        perpus.pinjamkan("M001", "B001", date(2026, 10, 28))
    except BukuSedangDipinjamError as exc:
        print("Ditolak:", exc)
    print("Jumlah riwayat:", len(perpus.daftar_peminjaman()))

    perpus.kembalikan("P001", date(2026, 11, 2))
    print("Sesudah kembali:", perpus.tersedia("B001"))
    digital = perpus.pinjamkan("M001", "E001", date(2026, 10, 27))
    print("E-book jatuh tempo:", digital.tanggal_jatuh_tempo)
    perpus.simpan()

    dimuat = Perpustakaan(simpanan)
    dimuat.muat()
    print("Dimuat:", len(dimuat.daftar_buku()), "buku,",
          len(dimuat.daftar_peminjaman()), "riwayat")
    print("Status:", [(p.id_pinjam, p.aktif) for p in dimuat.daftar_peminjaman()
    ])
```

Keluaran

```
Pinjam: P001 jatuh tempo 2026-11-10
B001 tersedia: False
Ditolak: Buku masih dipinjam.
Jumlah riwayat: 1
Sesudah kembali: True
E-book jatuh tempo: 2026-11-03
Dimuat: 2 buku, 2 riwayat
Status: [('P001', False), ('P002', True)]
```

Jumlah riwayat tetap satu setelah percobaan peminjaman ganda, sehingga penolakan tidak membuat catatan tambahan. Setelah P001 dikembalikan, B001 tersedia kembali. Transaksi e-book berikutnya mendapat ID P002 dan jatuh tempo tujuh hari setelah tanggal pinjam. Pemuatan ulang mempertahankan kedua catatan beserta statusnya: P001 selesai, P002 aktif.

Memeriksa Kontrak dengan Assertion

Keluaran membantu pembaca menelusuri alur, tetapi assertion dapat memeriksa syarat secara langsung. Simpan pemeriksaan berikut sebagai `cek_kontrak.py` di folder utama. Contoh ini menilai relasi setelah simpan–muat dan memastikan pengembalian yang ditolak tidak mengubah transaksi. Assertion di sini dipakai untuk pengujian; validasi masukan aplikasi tetap menggunakan exception, sebab assertion dapat dinonaktifkan oleh opsi interpreter.

cek_kontrak.py

```python
from datetime import date
from pathlib import Path
from tempfile import TemporaryDirectory

from perpustakaan import Anggota, BukuCetak, Perpustakaan, PenyimpananJSON
from perpustakaan.kesalahan import DataTidakValidError


with TemporaryDirectory() as folder:
    simpanan = PenyimpananJSON(Path(folder) / "data.json")
    perpus = Perpustakaan(simpanan)
    perpus.tambah_buku(BukuCetak("B001", "Dasar OOP"))
    perpus.tambah_anggota(Anggota("M001", "Ani"))
    pinjam = perpus.pinjamkan("M001", "B001", date(2026, 10, 27))

    try:
        perpus.kembalikan("P001", date(2026, 10, 26))
    except DataTidakValidError:
        pass
    else:
        raise AssertionError("Tanggal kembali seharusnya ditolak.")
    assert pinjam.aktif and pinjam.tanggal_kembali is None
    assert len(perpus.daftar_peminjaman()) == 1

    perpus.simpan()
    dimuat = Perpustakaan(simpanan)
    dimuat.muat()
    riwayat = dimuat.daftar_peminjaman()
    assert riwayat[0].buku is dimuat.daftar_buku()[0]
    assert riwayat[0].anggota is dimuat.daftar_anggota()[0]
    assert not dimuat.tersedia("B001")

    dimuat.kembalikan("P001", date(2026, 11, 2))
    berikut = dimuat.pinjamkan("M001", "B001", date(2026, 11, 3))
    assert berikut.id_pinjam == "P002"

print("Kontrak pengembalian, relasi, dan nomor transaksi terpenuhi.")
```

Keluaran

```
Kontrak pengembalian, relasi, dan nomor transaksi terpenuhi.
```

Pemeriksaan identitas rujukan memakai `is`: object buku pada transaksi hasil muat harus sama dengan object yang ada di daftar buku layanan hasil muat. Kesamaan judul saja belum membuktikan hubungan tersebut. Pemeriksaan ID P002 memastikan penghitung ikut disimpan, bukan kembali ke satu saat aplikasi dibuka ulang.

Tabel 12.3. Skenario tambahan dan hasil yang harus diperiksa.

Percobaan	Hasil yang diharapkan
Kode buku atau ID anggota ganda	Ditolak; object yang sudah terdaftar tetap sama.
Anggota atau buku tidak terdaftar	Ditolak; riwayat dan nomor berikutnya tidak berubah.
Mengembalikan sebelum tanggal pinjam	Ditolak; transaksi tetap aktif.
Mengembalikan transaksi selesai	Ditolak; tanggal kembali sebelumnya tetap tersimpan.
File JSON berisi teks yang tidak sah	Pemuatan gagal dengan pesan; simpanan tidak ditimpa data kosong.
JSON sah tetapi rujukan ID hilang	Pemuatan gagal; keadaan lama di memori tetap utuh.
Dua catatan aktif untuk satu buku	Data ditolak saat dimuat.
Gagal menulis file sementara	Kesalahan penyimpanan dilaporkan; file tujuan lama belum diganti.

Urutan Pemeriksaan sebelum Membuat GUI

1. Lengkapi file package, `main.py`, dan file demo pada folder yang benar.
2. Jalankan demo konsep dan cocokkan keluarannya dengan contoh pada bab ini.
3. Jalankan `python demo_integrasi.py` serta `python cek_kontrak.py`.
4. Coba menu terminal dari data baru, simpan dengan pilihan 0, lalu buka kembali.
5. Periksa skenario penolakan. Catat keadaan sebelum dan sesudah operasi yang gagal.
6. Gunakan salinan file data untuk mencoba kasus kerusakan JSON atau rujukan ID.

Setelah langkah tersebut terpenuhi, model, layanan, dan penyimpanan sudah dapat dipakai oleh GUI. Pada Bab 13, tombol dan formulir akan menggantikan input menu sambil tetap memanggil operasi layanan yang sama.

12.8 Rangkuman

- Implementasi menerjemahkan tanggung jawab dan kontrak rancangan menjadi perilaku yang dapat dijalankan dan diperiksa.
- Property menjaga perubahan data: identitas hanya dibaca, sedangkan judul dan nama diperbarui melalui setter yang melakukan validasi.
- Inheritance dipakai untuk jenis buku; association menghubungkan transaksi dengan anggota serta buku; layanan memiliki kumpulan catatan peminjamannya.
- Polymorphism memungkinkan perhitungan jatuh tempo memanggil operasi yang sama untuk jenis buku yang berbeda.
- Module memisahkan model, exception, penyimpanan, layanan, dan tampilan. Arah import mengikuti tanggung jawab tersebut.
- JSON menyimpan data sederhana serta ID rujukan. Hubungan object dibangun kembali setelah buku dan anggota dimuat.
- Pemuatan divalidasi pada keadaan sementara sebelum mengganti keadaan lama. Penyimpanan menulis file sementara sebelum mengganti file tujuan.
- Aturan lintas-object dijalankan melalui layanan. Kegagalan transaksi tidak boleh meninggalkan perubahan sebagian.
- Pemeriksaan integrasi perlu mencakup hasil normal, penolakan, kelanjutan ID, dan hubungan object setelah simpan–muat.

12.9 Kuis Teoritis

Pilihan Ganda

Pilihlah satu jawaban yang paling tepat berdasarkan implementasi dalam bab ini.

1. Kode buku disediakan sebagai property hanya-baca karena ...

   a. judul buku tidak boleh diubah
   b. kode menjadi identitas tetap dan kunci koleksi
   c. semua attribute Python harus tidak dapat diubah
   d. JSON tidak dapat menyimpan kode berupa teks

2. Jika setter judul menerima teks berisi spasi saja, hasilnya adalah ...

   a. judul lama dihapus sebelum kesalahan dilaporkan
   b. judul diganti dengan satu spasi
   c. exception dilaporkan dan judul lama tetap ada
   d. object buku otomatis dihapus

3. Pemanggilan `buku.lama_pinjam()` pada constructor peminjaman merupakan contoh ...

   a. polymorphism
   b. pembacaan file
   c. duplikasi seluruh object
   d. penghapusan inheritance

4. Pemeriksaan bahwa B001 belum mempunyai peminjaman aktif lain dilakukan oleh ...

   a. menu saja
   b. `Anggota.info()`
   c. encoder JSON secara otomatis
   d. layanan `Perpustakaan`

5. Setelah `daftar_buku()` menghasilkan list baru, pemanggilan `clear()` pada list itu akan ...

   a. menghapus semua buku pada layanan
   b. mengosongkan list hasil tanpa menghapus dictionary layanan
   c. menghapus file JSON secara otomatis
   d. mengubah semua kode buku

6. Cara menyimpan hubungan peminjaman dengan buku dalam JSON adalah ...

   a. menyalin method buku sebagai teks kode
   b. menyimpan alamat memori Python

c. menyimpan kode buku lalu menghubungkannya kembali saat dimuat
d. menyimpan hanya judul dan mengabaikan identitas

7. Alasan memakai keadaan sementara saat pemuatan adalah ...

a. mencegah kegagalan di tengah pemuatan meninggalkan data lama yang sudah berubah sebagian
b. membuat setiap kesalahan selalu diabaikan
c. menghindari kebutuhan validasi
d. mengganti semua ID dengan `id()`

8. Saat buku yang masih dipinjam diminta lagi, nomor berikutnya ...

a. selalu bertambah dua
b. selalu kembali ke satu
c. berubah menjadi kode anggota
d. tetap karena catatan baru belum berhasil dibuat

9. Jika JSON ada tetapi tidak dapat dibaca, menu terminal akan ...

a. menghapusnya tanpa memberi pesan
b. melaporkan kegagalan dan berhenti sebelum mengubah simpanan
c. otomatis menganggap semua buku tersedia lalu menyimpan
d. memuat hanya baris pertama tanpa pemeriksaan

10. Penyimpanan tanggal jatuh tempo di dalam transaksi bertujuan untuk ...

a. selalu mengikuti perubahan kebijakan terbaru pada riwayat lama
b. mengganti tanggal pinjam setiap kali file dibuka
c. mempertahankan kesepakatan jatuh tempo ketika transaksi dibuat
d. menghilangkan kebutuhan tanggal kembali

Uraian

1. Jelaskan mengapa validasi tanggal diletakkan pada `Peminjaman`, sedangkan pemeriksaan buku tersedia diletakkan pada `Perpustakaan`.
2. Telusuri apa yang berubah dan tidak berubah ketika peminjaman kedua untuk buku yang sama ditolak.
3. Jelaskan perbedaan inheritance buku dengan association antara peminjaman dan anggota. Gunakan hasil `demo_relasi.py` untuk mendukung jawaban.
4. Mengapa pemuatan JSON perlu membentuk buku dan anggota sebelum peminjaman? Apa yang harus dilakukan jika ID rujukannya tidak ditemukan?
5. Jika ditambahkan `BukuReferensi` dengan masa pinjam tiga hari, bagian mana yang harus ditambah dan bagian mana yang tetap dapat digunakan? Sertakan pemetaan JSON serta pilihan input dalam jawaban.

KUNCI PILIHAN GANDA

1 b; 2 c; 3 a; 4 d; 5 b; 6 c; 7 a; 8 d; 9 b; 10 c.

12.10 Praktik Mandiri

Latihan berikut melanjutkan proyek, bukan mengganti seluruh rancangan. Gunakan salinan proyek dan data khusus latihan. Bentuk latihan dengan syarat serta hasil yang dapat diperiksa membantu pembaca menilai perilaku programnya sendiri, sebagaimana pendekatan latihan pada [Edcorner Learning, tt].

Praktik 1: Menambah Jenis Buku

Tambahkan class `BukuReferensi` dengan penanda jenis `"referensi"` dan lama pinjam tiga hari. Perbarui ekspor package, pemetaan jenis ketika memuat, serta pilihan input menu. Gunakan seluruh validasi kode dan judul yang sudah tersedia pada induk.

- Hasil wajib: peminjaman pada 27 Oktober 2026 jatuh tempo pada 30 Oktober 2026.
- Simpan, muat ulang, lalu pastikan object kembali menjadi `BukuReferensi`, bukan `BukuCetak`.
- Buktikan bahwa rumus pembuatan jatuh tempo pada `Peminjaman` tidak perlu ditambah percabangan jenis.

Praktik 2: Menampilkan Peminjaman Terlambat

Tambahkan operasi layanan yang menerima tanggal pemeriksaan dan menghasilkan peminjaman yang masih aktif dengan tanggal jatuh tempo *sebelum* tanggal pemeriksaan. Jangan menyimpan flag `terlambat` tambahan karena status ini bergantung pada tanggal pemeriksaan.

- Hasil wajib: transaksi yang jatuh tempo 10 November belum terlambat pada 10 November, tetapi terlambat pada 11 November jika belum dikembalikan.
- Transaksi selesai tidak ditampilkan, walaupun tanggal pemeriksaan melewati jatuh temponya.
- Tambahkan menu laporan dan uji masukan tanggal yang tidak valid tanpa mengubah riwayat.

Praktik 3: Memeriksa Kegagalan Simpan–Muat

Buat pemeriksaan terpisah menggunakan folder sementara. Siapkan data sah, lalu uji salinan data dengan ID anggota hilang, dua transaksi aktif pada satu buku, nomor berikutnya yang sudah terpakai, dan JSON yang tidak sah. Periksa exception serta keadaan layanan sebelum dan sesudah setiap percobaan.

- Hasil wajib: seluruh data salah ditolak, keadaan lama tetap ada, dan file asli tidak ditimpa oleh data kosong.
- Tambahkan kasus simpan–muat yang berhasil untuk membuktikan bahwa pemeriksaan tidak hanya menolak semua file.
- Catat langkah, hasil yang diharapkan, hasil aktual, dan penyebab jika terjadi perbedaan.

Tabel 12.4. Rubrik praktik implementasi.

Aspek	Bobot	Bukti
Aturan dan hasil operasi	35%	Skenario normal serta penolakan menghasilkan keadaan yang tepat.
Pemakaian konsep OOP	25%	Tanggung jawab, property, relasi, dan polymorphism sesuai kebutuhan.
Integritas simpan–muat	25%	Identitas, relasi, jenis object, tanggal, dan nomor transaksi bertahan.
Keterbacaan dan pemeriksaan	15%	File tersusun jelas, hasil bisa diulang, dan laporan menjelaskan perilaku yang diperiksa.

Daftar Pustaka

Sumber yang dirujuk dalam bab ini.

[Edcorner Learning, tt] Edcorner Learning (t.t.). *Python Object Oriented Programming Exercises: Become a Pro Developer*. Penerbit tidak tercantum. Tahun terbit tidak tercantum pada salinan digital yang digunakan.

[Lutz, 2025] Lutz, M. (2025). *Learning Python: Powerful Object-Oriented Programming*. O'Reilly Media, Sebastopol, CA, 6 edition.

[North, 2025] North, M. (2025). *Object-Oriented Programming in Python: Design and Build Programs Using Classes, Objects, and the Four Pillars of OOP*, volume 1 of *Think, Code, Create – Advanced Python Programming Series*. Northward House Publishing, Brisbane, Queensland, Australia.

[Wilson, 2024] Wilson, G. (2024). *Software Design by Example: A Tool-Based Introduction with Python*. CRC Press.